eg100.cpp

```
/*
assignment (make the following happen)
b1=new (pool()) Bulb;
*/
#include<iostream>
using namespace std;
class Bulb
{
private:
int wattage;
int price;
public:
explicit Bulb()
{
this->wattage=0;
this->price=10;
}
explicit Bulb(int wattage,int price):wattage(wattage),price(price){}
void set_wattage(int wattage)
{
this->wattage=wattage;
}
int get_wattage()
{
return this->wattage;
}
void  set_price(int price)
{
this->price=price;
}
int get_price()
{
return this->price;
}
};
template<typename T>
class memory_pool
{
private:
union node
{
 node(){}  ~node(){}
       T obj;
 node *next;
};
node *list;
size_t sz;
node *head;
```

```
public:
memory_pool(size_t sz)  // cold path / setup path
{
node *head;
this->sz=sz;
this->list=new node[sz]; // one time setup cost
// setting up linked list
head=nullptr;
for(int i=0;i<sz;++i)
{
this->list[i].next=head;
head=&(this->list[i]);
}
this->head=head;
}
void * acquire() // hot path
{
if(this->head==nullptr) return nullptr;
void *p=this->head;
head=head->next;
return p;
}
void release(T *p) // hot path
{
if(!p) return;
p->~T();
node *n;
n=reinterpret_cast<node *>(p);
n->next=head;
head=n;
}
bool is_available() // hot path
{
return this->head!=nullptr;
}
~memory_pool()
{
delete [] this->list;
}
};
int main()
{
memory_pool<Bulb>  pool(5);
Bulb *b1=nullptr;
Bulb *b2=nullptr;
b1=new (pool.acquire()) Bulb;
if(b1)
{
b1->set_wattage(100);
```

```
b1->set_price(50);
cout<<b1->get_wattage()<<endl;
cout<<b1->get_price()<<endl;
}
b2=new (pool.acquire()) Bulb(60,10);
if(b2)
{
cout<<b2->get_wattage()<<endl;
cout<<b2->get_price()<<endl;
}
pool.release(b1);
pool.release(b2);
return 0;
}
```

Thinking Machines Classroom Codes

eg101.cpp

```
#include<iostream>
using namespace std;
class arena  // arena with bump pointer
{
private:
size_t sz;
byte *head;
byte *buffer;
public:
arena(size_t sz):sz(sz)
{
this->buffer=new byte[sz];
this->head=this->buffer;
}
~arena()
{
delete [] buffer;
}
void * acquire(size_t sz)
{
if((((buffer+(this->sz-1))-head)+1)<sz) return nullptr;
void *p=head;
head+=sz;
return p;
}
bool is_available(size_t sz)
{
return (((buffer+(this->sz-1))-head)+1)>=sz;
}
void reset()
{
this->head=this->buffer;
}
};
class Bulb
{
private:
int wattage;
int price;
public:
explicit Bulb()
{
this->wattage=0;
this->price=10;
}
explicit Bulb(int wattage,int price):wattage(wattage),price(price){}
void set_wattage(int wattage)
{
```

```
this->wattage=wattage;
}
int get_wattage()
{
return this->wattage;
}
void  set_price(int price)
{
this->price=price;
}
int get_price()
{
return this->price;
}
};
int main()
{
arena a(100000);
Bulb *b1;
b1=new (a.acquire(sizeof(Bulb))) Bulb(100,10);
cout<<b1->get_wattage()<<endl;
cout<<b1->get_price()<<endl;
int *x;
x=new (a.acquire(sizeof(int)*10)) int[10];
for(int i=0;i<10;++i) x[i]=(i+1);
for(int i=0;i<10;++i) cout<<x[i]<<endl;
return 0;
}
```

eg102.cpp

```
#include<iostream>
using namespace std;
int main()
{
byte *buffer;
buffer=new byte[10];
size_t sz=10;
byte *head=buffer+5;
cout<<(((buffer+(sz-1))-head)+1);
return 0;
}
```

eg103.cpp

```
#include<iostream>
using namespace std;
void do_something(int e) noexcept
{
// some code
}
int main()
{
do_something(5);
return 0;
}
```

Thinking Machines Classroom Codes

eg104.cpp

```
#include<iostream>
using namespace std;
void do_something(int e)
{
if(e==10) throw invalid_argument(string("something is fishy"));
cout<<e<<endl;
}
int main()
{
try
{
do_something(10);
}catch(invalid_argument &e)
{
cout<<e.what()<<endl;
}
return 0;
}
```

eg105.cpp

```
#include<iostream>
using namespace std;
void do_something(int e)  noexcept
{
if(e==10) throw invalid_argument(string("something is fishy"));
cout<<e<<endl;
}
int main()
{
try
{
do_something(10);
}catch(invalid_argument &e)
{
cout<<e.what()<<endl;
}
return 0;
}
```

eg106.cpp

```
#include<iostream>
#include<new> // for hardware_destructive_interference_size
using namespace std;
int main()
{
cout<<hardware_destructive_interference_size<<endl;
alignas(64) int x;
alignas(hardware_destructive_interference_size) int y;
return 0;
}
```